DSA 107 · DATA SCIENCE AND ANALYTICS LABORATORY

Lab 4 Report

Getting Data from the Wild

World Bank country data and a web page table, from request to figure

Anıl Ege

Student ID: 250221021021

Instructor: Salih Sarp, PhD

22 October 2026

Fall 2026–2027

1. Objective

This fourth lab moved from data that waits at a web address to data that has to be asked for. The aim was to request data from a web service, turn its JSON answer into a table, repair the columns, filter out the rows that are not real observations, summarise what remains and draw one labelled figure - then read a second, messier table straight off a web page. My own test: check the status code, and look at the shape of the answer before flattening it into a table.

2. Data Description

Two sources this week. The first is the World Bank country API: a request to api.worldbank.org returns every country the Bank tracks, with its region, income group, capital city and the capital's coordinates. The answer is JSON - a list of two parts: a page of metadata (296 records in total) and the records themselves; each record is a dictionary whose region and incomeLevel fields are themselves small dictionaries. The second source is the Wikipedia page "List of countries by Human Development Index", read as HTML: the page holds 14 tables and the ranking is the long one (193 rows) with columns for rank, country, HDI value and annual growth.

Cleaning, in order: json_normalize opened the nested fields into dotted columns (296 rows × 18 columns); I kept the seven columns I needed and renamed region.value and incomeLevel.value to region and income_level; longitude and latitude arrived as text and were converted with pd.to_numeric(errors="coerce"), which leaves the 6 rows without coordinates as NaN. Finally, the 296 rows include aggregates such as "Euro area" or "World", which the Bank marks with region = "Aggregates"; a boolean mask removed them, and 217 real countries were left. The HDI table was left dirty on purpose: its growth column is text and its bottom rows carry footnote markers.

Property	World Bank country API	Wikipedia HDI page
Source	api.worldbank.org/v2/country (JSON endpoint)	en.wikipedia.org - HDI ranking page
Size	296 records; 18 columns after json_normalize	14 tables found; the 193-row ranking kept (5 columns)
One row	a country - or a regional aggregate	a country or territory's HDI rank
Key columns	name, capitalCity, longitude, latitude, region, income_level	Rank, Country or territory, HDI value, % annual growth
Quirks	nested fields (region.value); aggregates; 6 missing coordinates	percent stored as text; footnote markers (N/A[b]); NaN cells
After preprocessing	7 columns kept; coordinates numeric; aggregates removed - 217 countries	none yet - left dirty on purpose

Table 1. The two sources and their state.

3. Methodology

One Colab notebook, run top to bottom after a restart, using requests for the two downloads plus pandas 2.2.3 and seaborn (whitegrid) for the work. The steps, in order:

1. **Ask and check.** requests.get(url); response.status_code (200 = the server understood); .json() gives a list of two parts, and len() plus the first record were read before anything was flattened - the record's fields became the column plan.
2. **JSON to table.** pd.json_normalize on the records, because region and incomeLevel are dictionaries inside each record; the keep list trimmed 18 columns to 7; rename() removed the dots; pd.to_numeric(errors="coerce") made longitude and latitude numeric.

3. **Remove non-countries.** The boolean mask countries["region"] != "Aggregates" cut 296 rows to 217; isna().sum() showed 6 missing longitudes and latitudes.
4. **Summarise.** groupby(["region", "income_level"])["id"].count().unstack() counts countries per combination - region down the side, income across the top.
5. **Draw.** A seaborn scatter of longitude against latitude, hue=region, with axis labels and a title; the legend was moved outside the axes (bbox_to_anchor) so it cannot cover the dots.
6. **Read the second source.** requests.get(wiki_url, headers=...) identified me to the server; pd.read_html(StringIO(page.text)) returned 14 tables as a list; I picked tables[1] by shape (193 rows) and columns, then read dtypes and both ends. The notebook was pushed to my dsa107-labs repository.

4. Results

The request worked on the first try: status 200, and the metadata's 296 records all arrived, because per_page=400 asked for more than the total. json_normalize gave 296 rows × 18 columns; after keeping seven columns and dropping the aggregates, 217 countries remained, 6 of them without capital coordinates.

region	High income	Low income	Lower middle	Upper middle
East Asia & Pacific	15	1	8	13
Europe & Central Asia	40	-	3	15
Latin America & Caribbean	19	-	5	18
Middle East, North Africa, Afghanistan & Pakistan	8	3	7	5
North America	3	-	-	-
South Asia	-	-	4	2
Sub-Saharan Africa	1	21	20	6

Table 2. Number of countries for every region and income-level combination, unstacked (a dash: no country in that combination).

Table 2 shows where the countries sit: Europe & Central Asia has the most (58, 40 of them high income), North America the fewest (3, all high income), and Sub-Saharan Africa is the mirror image - 48 countries, 41 low or lower-middle income. The dashes are counts of zero, not missing data.

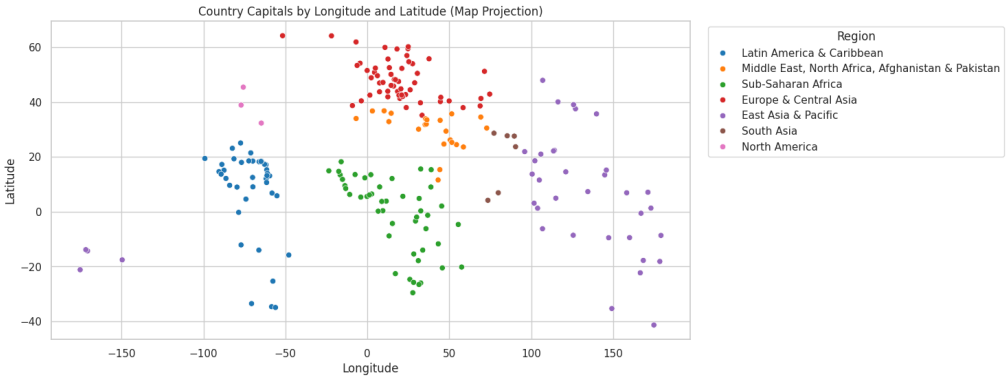


Figure 1. Capitals by longitude and latitude, coloured by world region.

Figure 1 draws the same 217 countries as a map: each dot is a capital city, and the clusters reproduce the continents - the Americas on the left, Europe and Africa in the middle, East Asia and the Pacific on the right. The colour adds the Bank's grouping: some of it is trivially geographic (Sub-Saharan Africa is a block), while the six countries without coordinates never appear at all.

The second source: read_html found 14 tables on the HDI page and the ranking is tables[1] (193 rows, five columns), from Iceland at 0.972 down to South Sudan at 0.388. The cleanliness fails in between: the growth column is text ("0.28%", "N/A[b]") and the change column carries brackets like "(2)".

5. Discussion

The lab's real lesson is that getting data is part of the analysis, not a step before it. The API answer was not a table: metadata plus a tree, flattened into 18 columns, a third of them names with dots. Two traps showed why the repairs matter: longitude and latitude arrived as text, and a scatter of text coordinates is not a map (my first figure was a wall of labels); and the 296 rows were not 296 countries, because the Bank tracks aggregates - any group mean on the raw list would have counted "World" as a country. The second source added its own warning: a web table is drawn for human eyes, so its numbers carry percent signs and its footnotes letters. What this does not show: the map draws capitals, not countries; six capitals never appear; and the HDI table was left dirty, so no comparison between the sources was attempted - joining them on the cleaned country name is the next step.

6. Reflection

Two things were new, and one was painful. First, reading JSON before flattening it: printing len(data), the metadata and one record made the structure obvious. Second, the figure - my first scatter came out as a wall of overlapping labels because longitude and latitude were still text; the fix only worked once it sat inside the plotting cell itself, not in a new cell above it. What I understand now: a status code and a look at the answer belong to every request, and "not every row is a country" is a cleaning thought that must come before any summary - without the mask, every number after it would have been wrong.

7. References

[1] World Bank Country API, api.worldbank.org/v2/country - the service read by the notebook.

[2] Wikipedia, HDI ranking page: en.wikipedia.org/wiki/List_of_countries_by_Human_Development_Index

[3] Waskom, M. (2021). seaborn: statistical data visualization. Journal of Open Source Software, 6(60), 3021.